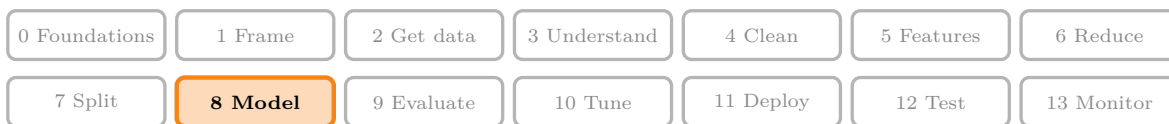


Backpropagation Through Time

Contents

1. Overview	2
2. Prerequisites	2
3. The setup: a many-to-one RNN	3
3.1 The data	3
3.2 The network	3
3.3 Forward propagation	4
4. What training needs: three derivatives	4
5. The gradient for W_o	4
6. The gradient for W_i	5
6.1 Three paths	5
6.2 The compact formula	6
6.3 The worked example	6
7. The gradient for W_h	7
8. The full training loop	8
9. Summary	10
10. Sources	10
11. Key terms	10

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Backpropagation ([Note 1019](#)).
- **Leads to:** Long-term dependency problem ([Note 1060](#)).

1. Overview

Key point: Backpropagation through time (BPTT) is ordinary backpropagation applied to the unfolded RNN. Because W_i and W_h are used at every time step, their gradient is a sum with one term per time step.

An RNN learns like any neural network: forward propagation gives a prediction and a loss, backpropagation gives the derivative of the loss with respect to every weight, and gradient descent updates the weights. In an RNN this backward pass runs over the network unfolded in time, so it is called **backpropagation through time (BPTT)**; Goodfellow §10.2). No special algorithm is needed: the chain rule on the unfolded graph is enough (Goodfellow §10.2.2).

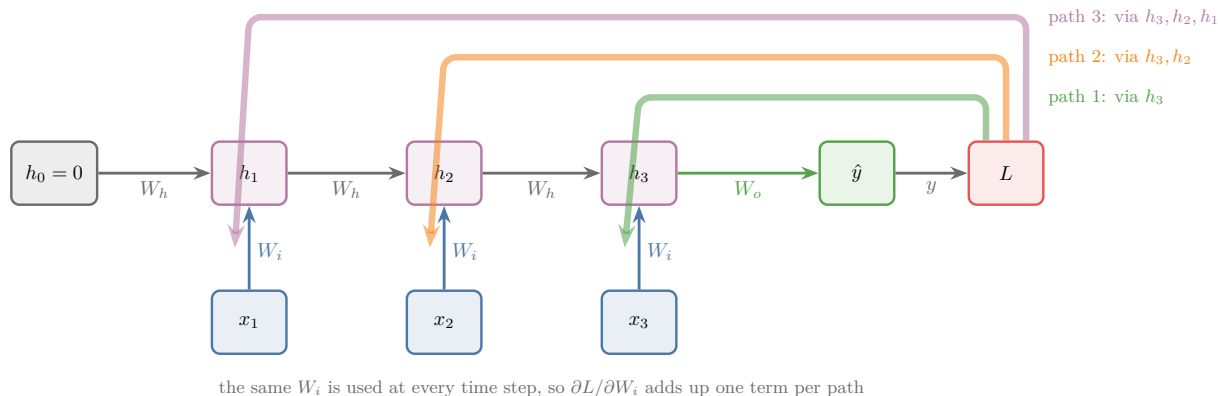


Figure 1: The RNN unfolded over three time steps. Loss L depends on W_i through three paths, one for each time step at which W_i is used. BPTT adds up the three terms

Figure 1 shows the one new idea. The same W_i enters at every time step, so the loss reaches it along several paths. The same holds for W_h .

2. Prerequisites

- The [RNN forward propagation Note](#): $h_t = \tanh(x_t W_i + h_{t-1} W_h)$, the unfolded network and the notation.
- The backpropagation Notes, [part 1](#) and [part 2](#): gradients as chain-rule products, and the update $w \leftarrow w - \eta \partial L / \partial w$.

- The [partial derivatives Note](#): the chain rule with several paths, where the contributions of the paths add up.
- The [loss functions Note](#): binary cross-entropy.

3. The setup: a many-to-one RNN

Key point: Three reviews of three words each, a sentiment of 1 or 0, one-hot vectors of 3 numbers, a recurrent layer of 3 nodes and a sigmoid output: three weight matrices W_i , W_h and W_o .

3.1 The data

Key point: A vocabulary of three words, so every word is a vector of 3 numbers and every review is a 3×3 table.

We use a sentiment analysis task: the input is a text, and the **target** (the output we predict) is its sentiment, 1 for positive and 0 for negative. The toy dataset has three reviews, each one **observation** (one record):

Review	Sentiment
cat mat rat	1
rat rat mat	1
mat mat cat	0

The vocabulary has three words, so one-hot encoding turns each into 3 numbers: cat = $[1, 0, 0]$, mat = $[0, 1, 0]$, rat = $[0, 0, 1]$. Each of the 3 positions is one input **feature** (one input variable). Review i is x_i , and its word at time step t is x_{it} .

The task is **many-to-one**: a sequence goes in, a single output comes out (see the [types of RNN Note](#)).

3.2 The network

Key point: W_i is 3×3 , W_h is 3×3 , W_o is 3×1 . We leave out the biases to keep the formulas short.

- **Input:** 3 numbers per time step.
- **Recurrent layer:** 3 nodes, with input weights W_i (3×3) and feedback weights W_h (3×3).
- **Output:** 1 sigmoid node, with weights W_o (3×1).

The three matrices hold 21 weights. The biases (3 in the recurrent layer, 1 in the output) are learned in exactly the same way as the weights, so we leave them out here.

3.3 Forward propagation

Key point: Three steps of the recurrent layer, then the output, then the loss.

For one review, with $h_0 = 0$ and $\tanh$ in the recurrent layer (as in the [forward propagation Note](#)):

$$h_1 = \tanh(x_{i1}W_i + h_0W_h), \quad h_2 = \tanh(x_{i2}W_i + h_1W_h), \quad h_3 = \tanh(x_{i3}W_i + h_2W_h)$$

$$\hat{y} = \sigma(h_3W_o), \quad L = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

The loss is binary cross-entropy (see the [loss functions Note](#)). The whole flow is: x_{i1} and h_0 give h_1 ; x_{i2} and h_1 give h_2 ; x_{i3} and h_2 give h_3 ; h_3 gives $\hat{y}$; $\hat{y}$ and y give L .

4. What training needs: three derivatives

Key point: Gradient descent updates W_i , W_h and W_o , so we need $\partial L / \partial W_i$, $\partial L / \partial W_h$ and $\partial L / \partial W_o$.

Training looks for the values of W_i , W_h and W_o that make L smallest. Gradient descent updates each one with the learning rate η :

$$W_i \leftarrow W_i - \eta \frac{\partial L}{\partial W_i}, \quad W_h \leftarrow W_h - \eta \frac{\partial L}{\partial W_h}, \quad W_o \leftarrow W_o - \eta \frac{\partial L}{\partial W_o}$$

We have the starting weights and η . So the whole job of BPTT is to compute these three derivatives. Backpropagation goes from the back to the front, so we start with W_o , nearest to the output.

5. The gradient for W_o

Key point: W_o is used once, so there is a single path: $\partial L / \partial W_o = (\partial L / \partial \hat{y})(\partial \hat{y} / \partial W_o)$. Nothing differs from an ANN.

$\partial L / \partial W_o$ asks: how much does the loss change if W_o changes a little? L depends on $\hat{y}$ only (y is fixed data), and $\hat{y}$ depends on h_3 and W_o . So there is one chain:

1. **In words:** follow the single path from L through $\hat{y}$ to W_o , multiplying the derivatives along it.
2. **Formula:**

$$\frac{\partial L}{\partial W_o} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_o} = h_3^\top (\hat{y} - y)$$

For a sigmoid output with binary cross-entropy, the two factors combine into $\hat{y} - y$ times the input of the output node, h_3 (derived in the [logistic regression gradient descent Note](#)).

3. **Example:** a network with **one** hidden node, so every weight is a single number: $w_i = 0.5$, $w_h = 0.8$, $w_o = 1.0$, input sequence $x = (1, 0, 1)$, target $y = 1$. Forward propagation gives

$$h_1 = \tanh(0.5) = 0.462, \quad h_2 = \tanh(0.8 \times 0.462) = 0.354, \quad h_3 = \tanh(0.5 + 0.8 \times 0.354) = 0.654$$

$$\hat{y} = \sigma(1.0 \times 0.654) = 0.658, \quad L = -\log 0.658 = 0.419$$

Then

$$\frac{\partial L}{\partial w_o} = h_3 (\hat{y} - y) = 0.654 \times (0.658 - 1) = 0.654 \times (-0.342) = -0.224$$

6. The gradient for W_i

Key point: W_i is used at all three time steps, so the loss reaches it along three paths. The gradient is the sum of the three path products.

6.1 Three paths

Key point: h_3 depends on W_i directly, and also through h_2 , which depends on W_i directly and through h_1 .

Follow the dependencies backwards from L :

- L depends on $\hat{y}$; $\hat{y}$ depends on h_3 and W_o .
- h_3 depends on four things: x_{i3} , W_i , h_2 and W_h .
- h_2 depends on x_{i2} , W_i , h_1 and W_h .
- h_1 depends on x_{i1} , W_i , h_0 and W_h .

So W_i affects L in three ways, the three coloured paths of Figure 1. By the chain rule with several paths (section 6 of the [partial derivatives Note](#)), the derivative is the sum of the three path products:

$$\frac{\partial L}{\partial W_i} = \underbrace{\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial W_i}}_{\text{path 1}} + \underbrace{\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_i}}_{\text{path 2}} + \underbrace{\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_i}}_{\text{path 3}}$$

In each path, the last factor $\partial h_t / \partial W_i$ is the **immediate** derivative: how h_t changes with W_i when h_{t-1} is held fixed, so only the direct use of W_i at step t counts (Pascanu et al. 2013 use the same convention). The other uses of W_i are covered by the other paths.

6.2 The compact formula

Key point: One term per time step: $\partial L / \partial W_i$ is a sum over $j = 1, \dots, T$ of $(\partial L / \partial \hat{y})(\partial \hat{y} / \partial h_j)(\partial h_j / \partial W_i)$, where T is the number of time steps.

Three time steps give three terms; a 10-word review would give 10. Writing them all out is not practical, so we summarise with a sum over the time steps j :

$$\frac{\partial L}{\partial W_i} = \sum_{j=1}^T \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_j} \frac{\partial h_j}{\partial W_i}$$

The middle factor $\partial \hat{y} / \partial h_j$ hides a chain. $\hat{y}$ does not use h_1 directly: it uses h_3 , which uses h_2 , which uses h_1 . Expanding it gives back the paths of section 6.1:

- $j = 1$: $\partial \hat{y} / \partial h_1 = (\partial \hat{y} / \partial h_3)(\partial h_3 / \partial h_2)(\partial h_2 / \partial h_1)$, which is path 3.
- $j = 2$: $\partial \hat{y} / \partial h_2 = (\partial \hat{y} / \partial h_3)(\partial h_3 / \partial h_2)$, which is path 2.
- $j = 3$: $\partial \hat{y} / \partial h_3$ as it is, which is path 1.

Extra: Goodfellow §10.2.2 makes the bookkeeping exact with dummy variables: each time step t gets its own copy $W^{(t)}$ of the shared matrix, used only at that step. The gradient of the shared matrix is then the sum of the gradients of its copies. The Notebook does exactly this on a real IMDB movie review cut to 100 words, with an **Embedding** layer, a **SimpleRNN** of 16 nodes and a sigmoid output: the 100 per-step gradients for W_h add up to Keras' own gradient, to within 5×10^{-8} .

6.3 The worked example

Key point: In the one-node example, the three terms are -0.196 , 0 and -0.086 , so $\partial L / \partial w_i = -0.282$.

1. **In words:** for each time step, multiply the error at the output by the derivatives that carry it back to that step, then by the immediate derivative of that step.
2. **Formula:** with one node every factor is a number. Writing $d_t = 1 - h_t^2$ for the slope of $\tanh$ at step t (see the [activation functions Note](#)):

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} = (\hat{y} - y) w_o, \quad \frac{\partial h_t}{\partial h_{t-1}} = d_t w_h, \quad \frac{\partial h_t}{\partial w_i} = d_t x_t$$

3. **Example:** with the numbers of section 5, $(\hat{y} - y) w_o = -0.342$, $d_3 = 0.572$, $d_2 = 0.875$, $d_1 = 0.786$, so $\partial h_3 / \partial h_2 = 0.572 \times 0.8 = 0.457$ and $\partial h_2 / \partial h_1 = 0.875 \times 0.8 = 0.700$.

$$\text{path 1} = -0.342 \times 0.572 \times 1 = -0.196$$

$$\text{path 2} = -0.342 \times 0.457 \times 0.875 \times 0 = 0$$

$$\text{path 3} = -0.342 \times 0.457 \times 0.700 \times 0.786 \times 1 = -0.086$$

$$\frac{\partial L}{\partial w_i} = -0.196 + 0 - 0.086 = -0.282$$

Path 2 is 0 because the second input is $x_2 = 0$: w_i had no effect at that step. TensorFlow's automatic gradient gives the same -0.282 (Notebook).

7. The gradient for W_h

Key point: W_h is also used at every time step, so its gradient has the same form: a sum over the time steps, with $\partial h_j / \partial W_h$ as the last factor.

W_h , the feedback weight, appears in the formula of every hidden state. So, exactly as for W_i :

- L depends on $\hat{y}$; $\hat{y}$ depends on W_o and h_3 ;
- h_3 depends on x_{i3} , W_i , h_2 and W_h ; h_2 on x_{i2} , W_i , h_1 and W_h ; h_1 on x_{i1} , W_i , h_0 and W_h .

There are again three paths, one through each use of W_h , and the same compact sum. Only the immediate derivative changes: at step t , W_h multiplies h_{t-1} , so the immediate derivative carries h_{t-1} where the one for W_i carried x_t .

1. **In words:** same paths, same factors; the last factor uses the previous hidden state instead of the input.
2. **Formula** (one node):

$$\frac{\partial h_t}{\partial w_h} = d_t h_{t-1}$$

3. **Example:**

$$\text{path 1} = -0.342 \times 0.572 \times 0.354 = -0.069$$

$$\text{path 2} = -0.342 \times 0.457 \times 0.875 \times 0.462 = -0.063$$

$$\text{path 3} = -0.342 \times 0.457 \times 0.700 \times 0.786 \times 0 = 0$$

$$\frac{\partial L}{\partial w_h} = -0.069 - 0.063 + 0 = -0.132$$

Path 3 is 0 because $h_0 = 0$: at the first step there is no previous state for w_h to act on.

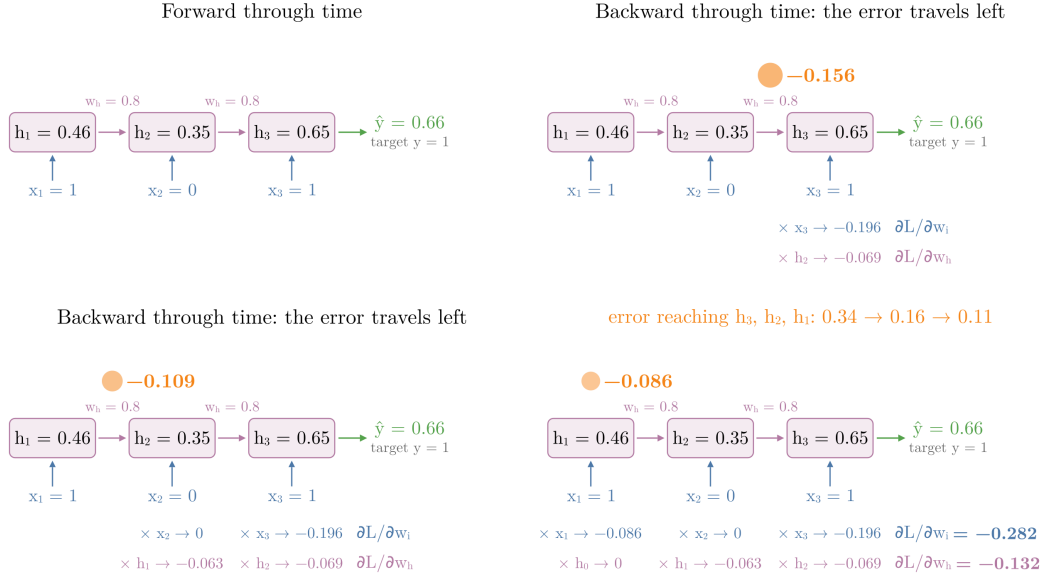


Figure 2: BPTT on the one-node example as one backward walk. The orange disc is the error reaching each hidden state; at each step it is multiplied by the tanh slope, drops one term into $\partial L/\partial w_i$ (times x_t) and one into $\partial L/\partial w_h$ (times h_{t-1}), then is multiplied by w_h to reach the step before

Figure 2 runs sections 6.3 and 7 together, from right to left. Watch the disc shrink: the error reaching h_3, h_2 and h_1 is 0.342, 0.156 and 0.109, because each step back multiplies it by $d_t w_h$ (0.457, then 0.700), and both factors are below 1 here. The further back a time step, the smaller the error that reaches it; the [problems with RNN Note](#) follows this effect over long sequences.

Extra: With matrices, the same computation runs as a loop backwards in time (Goodfellow §10.2.2, eq. 10.21 and 10.26, here in the row-vector form of Keras). Start with the error at the last hidden state, $\delta = (\hat{y} - y) W_o^T$. Then for $t = T$ down to 1:

1. pass it through tanh: $a = \delta \odot (1 - h_t^2)$, element by element;
2. add step t 's terms: $\partial L/\partial W_i += x_t^T a$ and $\partial L/\partial W_h += h_{t-1}^T a$;
3. move the error one step back: $\delta = a W_h^T$.

In the Notebook this loop, written in NumPy for the 3-node network on “cat mat rat”, matches TensorFlow’s gradients to within 4×10^{-8} .

8. The full training loop

Key point: Forward through time, loss, backward through time, update; then the next review. Repeat until the loss stops falling.

1. Start with initial values of W_i, W_h and W_o .
2. Take the first review. Send the first word and compute h_1 , the second word to get h_2 , the third word to get h_3 ; then compute $\hat{y}$ and the loss.

3. Compute the three derivatives with BPTT, and update the three matrices with gradient descent.
4. Repeat with the next review, using the new weights. Continue, epoch after epoch, until the weights reach the minimum of the loss.

Python: One BPTT update by hand, for the one-node example; TensorFlow's GradientTape gives the same numbers.

```
import tensorflow as tf
w_i, w_h, w_o = (tf.Variable(v) for v in (0.5, 0.8, 1.0))
with tf.GradientTape() as tape:
    h = 0.0
    for x_t in [1.0, 0.0, 1.0]:          # forward through time
        h = tf.tanh(w_i * x_t + w_h * h)
    loss = -tf.math.log(tf.sigmoid(w_o * h)) # target y = 1
    grads = tape.gradient(loss, [w_i, w_h, w_o])
    # [-0.282, -0.132, -0.224]
    for w, g in zip([w_i, w_h, w_o], grads):
        w.assign_sub(0.1 * g)             # gradient descent step
```

Run on the three toy reviews with gradient descent (learning rate 0.5, all three reviews averaged per update), the NumPy BPTT of the Notebook brings the mean loss from 0.809 to 0.004 in 300 epochs. The predictions become 0.997, 0.997 and 0.006 for targets 1, 1 and 0.

Compared with backpropagation in an ANN, the only new point is the unfolding in time, which turns one shared weight into several uses and its derivative into a sum.

9. Summary

Weight	Used at	Paths from L	Gradient
W_o	the last step only	1	$(\partial L / \partial \hat{y})(\partial \hat{y} / \partial W_o)$
W_i	every time step	T	$\sum_j (\partial L / \partial \hat{y})(\partial \hat{y} / \partial h_j)(\partial h_j / \partial W_i)$
W_h	every time step	T	$\sum_j (\partial L / \partial \hat{y})(\partial \hat{y} / \partial h_j)(\partial h_j / \partial W_h)$

- BPTT is backpropagation on the RNN unfolded in time.
- A weight used at several time steps reaches the loss along several paths; its gradient is the sum of the path products.
- $\partial \hat{y} / \partial h_j$ is itself a chain $\partial h_T / \partial h_{T-1} \cdots \partial h_{j+1} / \partial h_j$: the further back the step, the longer the chain.
- After BPTT, gradient descent updates W_i , W_h and W_o as in any network.

10. Sources

Built from

- CampusX, “How Backpropagation works in RNN | Backpropagation Through Time”, YouTube, <https://www.youtube.com/watch?v=OvCz1acvt-k>

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 10, §10.2 (recurrent neural networks, the name back-propagation through time) and §10.2.2 (computing the gradient in a recurrent neural network). deeplearning-book.org/contents/rnn.html.
- Pascanu, R., Mikolov, T. and Bengio, Y. (2013). On the Difficulty of Training Recurrent Neural Networks. ICML 2013. arXiv:1211.5063. §2 (the gradient as a sum of temporal contributions; the “immediate” partial derivative).

11. Key terms

Term	Meaning
Backpropagation through time (BPTT)	Backpropagation applied to an RNN unfolded in time
Observation	One record of the data, here one review
Feature	An input variable; here one of the 3 positions of a word vector
Target	The output we predict, here the sentiment
Many-to-one	An RNN task with a sequence as input and a single output
Path (in the chain rule)	One route through the computation from the loss to a weight; the derivative is the sum over all paths

Term	Meaning
Immediate derivative	The derivative of h_t with respect to a weight with h_{t-1} held fixed: only the weight's direct use at step t
Dummy copy $W^{(t)}$	A copy of a shared weight used only at time step t ; the gradient of the shared weight is the sum over the copies